

MariaDB Database Backup and Synchronization with Amazon S3

Description

GOAL: To move/sync the backups taken in server to the S3 bucket.

1. Create an IAM Role which allows access to S3, and attach with your AWS instance.
2. Login to the console (CLI) of the server and configure a MariaDB server.
3. Write a shell script for taking up the backups of databases to a directory.
4. Configure AWS CLI within the server and synchronize the backup directory with any of the AWS S3 buckets.
5. Verify the backups in S3.

1. Create IAM Role with S3 Access and Attach It to EC2

An IAM role named **backup-mariadb** was created to provide the EC2 instance with controlled access to Amazon S3.

Instead of providing full S3 permissions, a custom least-privilege policy was configured for the specific backup bucket.

The S3 bucket used for this task was:

`mariadb-backup123`

The policy allowed the EC2 instance to:

- List the contents of `mariadb-backup123`
- Upload objects to the bucket
- Download objects from the bucket
- Delete objects from the bucket

The policy used was:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListBackupBucket",
```

```

    "Effect": "Allow",
    "Action": "s3:ListBucket",
    "Resource": "arn:aws:s3:::mariadb-backup123"
  },
  {
    "Sid": "ManageBackupObjects",
    "Effect": "Allow",
    "Action": [
      "s3:PutObject",
      "s3:GetObject",
      "s3:DeleteObject"
    ],
    "Resource": "arn:aws:s3:::mariadb-backup123/*"
  }
]
}

```

The backup-mariadb role was then attached to the EC2 instance.

The role-based access was verified from the EC2 instance using:

```
aws s3 ls s3://mariadb-backup123
```

The command completed without an AccessDenied error. Since the bucket was initially empty, no objects were displayed.

The command:

```
aws s3 ls
```

returned AccessDenied because the role was not given `s3:ListAllMyBuckets`. This was intentional because the role was configured with minimum access rather than permission to list all S3 buckets.

2. Configure MariaDB Server

The EC2 instance was running **Red Hat Enterprise Linux 10.2**.

MariaDB client components were initially installed, but the MariaDB server package was not present. The MariaDB server was installed using:

```
sudo dnf install mariadb-server -y
```

The MariaDB service was enabled and started using:

```
sudo systemctl enable --now mariadb
```

The service status was checked to confirm that MariaDB was running.

The MariaDB console was accessed using:

```
sudo mariadb
```

A test database named **backupdb** was created:

```
CREATE DATABASE backupdb;
```

The database was selected:

```
USE backupdb;
```

A table named **employees** was created:

```
CREATE TABLE employees (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    department VARCHAR(100)  
);
```

Sample data was inserted:

```
INSERT INTO employees (name, department)  
VALUES  
( 'Sanjid', 'DevOps' ),  
( 'Rahul', 'AWS' ),  
( 'Aisha', 'Linux' );
```

The data was verified using:

```
SELECT * FROM employees;
```

This provided test data that was used to verify the MariaDB backup.

3. Create Database Backup Directory and Shell Script

A directory named **backup.db** was created in the EC2 user's home directory to store the database backups.

The directory was:

```
/home/ec2-user/backup.db
```

A manual database dump was created using:

```
sudo mariadb-dump backupdb > ~/backup.db/backupdb.sql
```

The generated backup was verified using:

```
ls -lh ~/backup.db/
```

The backup file was approximately **2.1 KB**.

The contents of the backup were checked using:

```
head -20 ~/backup.db/backupdb.sql
```

The output confirmed that the file contained MariaDB dump information and the structure of the employees table.

A shell script named **backup.sh** was created in the EC2 user's home directory.

The script was:

```
#!/bin/bash
```

```
BACKUP_DIR="/home/ec2-user/backup.db"
```

```
TIMESTAMP=$(date +%Y-%m-%d_%H-%M-%S)
```

```
BACKUP_FILE="$BACKUP_DIR/backupdb_${TIMESTAMP}.sql"
```

```
mkdir -p "$BACKUP_DIR"
```

```
sudo mariadb-dump backupdb > "$BACKUP_FILE"
```

```
if [ $? -eq 0 ]; then
```

```
    echo "Backup successful: $BACKUP_FILE"
```

```
else
```

```
    echo "Backup failed"
```

```
    exit 1
```

```
fi
```

The script was given execute permission:

```
chmod +x ~/backup.sh
```

The script was executed:

```
~/backup.sh
```

It successfully generated a timestamped backup file:

```
backupdb_2026-09-21_05-52-38.sql
```

The backup directory contained:

```
backup.db/  
├─ backupdb.sql  
└─ backupdb_2026-09-21_05-52-38.sql
```

4. Configure AWS CLI and Synchronize Backups with S3

AWS CLI was installed and configured on the EC2 instance.

No AWS access keys were manually configured on the server. The AWS CLI obtained permissions through the **backup-mariadb IAM role attached to the EC2 instance**.

The local backup directory was synchronized with the S3 bucket using:

```
aws s3 sync ~/backup.db/ s3://mariadb-backup123/
```

The backup files from the local backup.db directory were successfully uploaded to:

```
s3://mariadb-backup123/
```

5. Verify the Backups in S3

The uploaded objects were verified from the EC2 instance using:

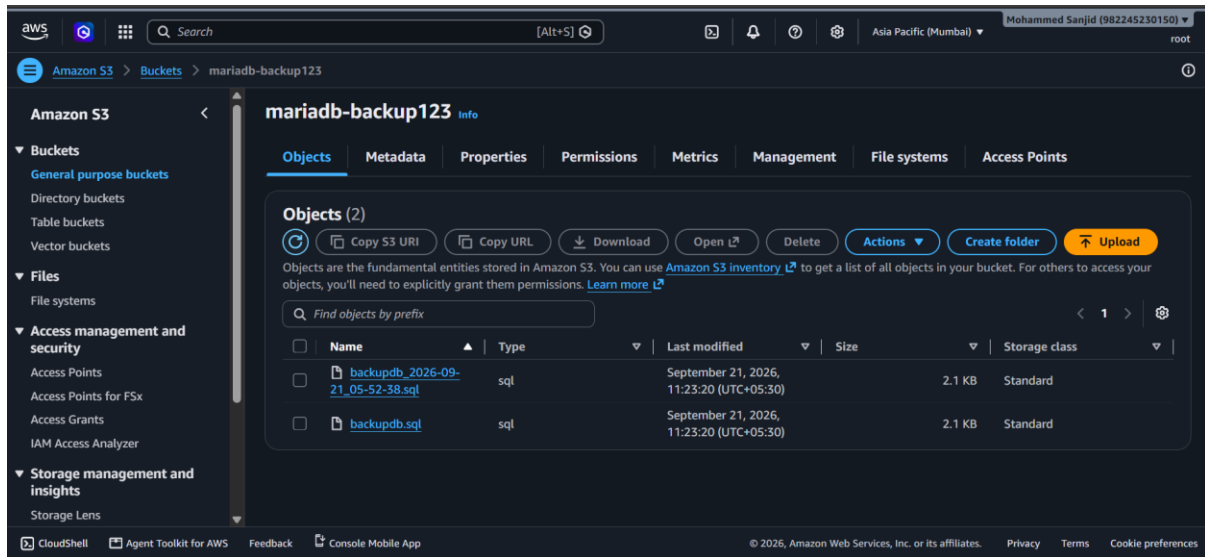
```
aws s3 ls s3://mariadb-backup123/
```

The output showed:

```
2026-09-21 05:53:20      2100 backupdb.sql  
2026-09-21 05:53:20      2100 backupdb_2026-09-21_05-52-38.sql
```

The Amazon S3 console was also checked. The bucket **mariadb-backup123** displayed both SQL backup files successfully.

Screenshot



Conclusion

The MariaDB backup and S3 synchronization task was successfully completed.

A MariaDB server was configured on the RHEL EC2 instance and a test database named backupdb was created with sample data. A shell script was developed to generate timestamped SQL backups in the /home/ec2-user/backup.db directory.

An IAM role named backup-mariadb was attached to the EC2 instance with minimum S3 permissions for the mariadb-backup123 bucket. AWS CLI used the attached IAM role to authenticate without storing AWS access keys on the server.

The backup directory was synchronized with Amazon S3 using `aws s3 sync`, and the generated SQL backup files were successfully verified in the **mariadb-backup123** S3 bucket.